



C++ - Módulo 04

Polimorfismo de subtipos, clases abstractas y Interfaces

Resumen: Este documento contiene los ejercicios del Módulo 04 de los módulos de C++.

Versión: 13.1

Contenido

I	Introducción	2
II	Reglas generales	3
III	Instrucciones de IA	6
IV	Ejercicio 00: Polimorfismo	8
V	Ejercicio 01: No quiero incendiar el mundo	10
VI	Ejercicio 02: Clase abstracta	12
VII	Ejercicio 03: Interfaz y resumen	13
VIII	Presentación y evaluación por pares	17

Capítulo I

Introducción

C++ es un lenguaje de programación de propósito general creado por Bjarne Stroustrup como una extensión del lenguaje de programación C, o "C con clases" (fuente: [Wikipedia](#)).

El objetivo de estos módulos es introducirte en la Programación Orientada a Objetos. Este será el punto de partida de tu experiencia con C++. Muchos lenguajes se recomiendan para aprender programación orientada a objetos, pero hemos elegido C++ porque deriva de tu viejo amigo C. Dado que C++ es un lenguaje complejo, y para simplificar las cosas, tu código cumplirá con el estándar C++98.

Sabemos que el C++ moderno difiere significativamente en muchos aspectos. Por lo tanto, si quieres convertirte en un desarrollador competente de C++, ¡de ti depende continuar tu camino más allá de los 42 Common Core!

Capítulo II

Reglas generales

Compilando

- Compila tu código con `c++` y los indicadores `-Wall -Wextra -Werror`
- Su código aún debería compilarse si agrega el indicador `-std=c++98`

Convenciones de formato y nomenclatura

- Los directorios de ejercicios se nombrarán de esta manera: `ex00`, `ex01`, ... , `exn`
 - Nombra tus archivos, clases, funciones, funciones miembro y atributos según sea necesario en las directrices.
 - Escriba los nombres de clase en formato Mayúsculas y minúsculas . Los archivos que contienen código de clase siempre se nombrarán según el nombre de la clase.
Por ejemplo: `ClassName.hpp/ClassName.h`, `ClassName.cpp` o `ClassName.hpp`. Si tiene un archivo de encabezado que contiene la definición de la clase "BrickWall", que representa un muro de ladrillos, su nombre será `BrickWall.hpp`.
 - A menos que se especifique lo contrario, cada mensaje de salida debe terminar con un carácter de nueva línea y mostrarse en la salida estándar.
- ¡ Adiós, Norminette! No se impone ningún estilo de programación en los módulos de C++. Puedes seguir tu favorito. Pero recuerda que el código que tus pares evaluadores no pueden entender es código que no pueden calificar. Haz todo lo posible por escribir código limpio y legible.

Permitido/Prohibido

Ya no estás programando en C. ¡Hora de usar C++! Por lo tanto:

- Puedes usar casi todo lo de la biblioteca estándar. Por lo tanto, en lugar de limitarte a lo que ya sabes, sería inteligente usar las versiones C++ de las funciones de C que conoces siempre que sea posible.

Sin embargo, no puedes usar ninguna otra biblioteca externa. Esto significa que C++11 (y sus derivados) y las bibliotecas de Boost están prohibidas. Las siguientes funciones también están prohibidas: `*printf()`, `*alloc()` y `free()`. Si las usas, tu calificación será 0 y punto.

- Tenga en cuenta que, a menos que se indique explícitamente lo contrario, el uso del espacio de nombres `<ns_name>` y Las palabras clave de amistad están prohibidas. De lo contrario, tu calificación será -42.
- Solo se permite usar el STL en los módulos 08 y 09. Esto significa: no se permiten contenedores (vectoriales, de lista, de mapa, etc.) ni algoritmos (cualquier cosa que requiera incluir el encabezado `<algorithm>`) hasta entonces. De lo contrario, su calificación será de -42.

Algunos requisitos de diseño

- La pérdida de memoria también ocurre en C++. Cuando se asigna memoria (mediante el nuevo palabra clave), debe evitar fugas de memoria.
- Desde el Módulo 02 hasta el Módulo 09, sus clases deben estar diseñadas en el modelo Ortodoxo. Forma canónica, salvo que se indique explícitamente lo contrario.
- Cualquier implementación de función colocada en un archivo de encabezado (excepto las plantillas de función) significa 0 para el ejercicio.

Debes poder usar cada encabezado independientemente. Por lo tanto, deben incluir todas las dependencias necesarias. Sin embargo, debes evitar el problema de la doble inclusión añadiendo protecciones de inclusión. De lo contrario, tu calificación será 0.

Léame

- Puedes agregar archivos adicionales si lo necesitas (por ejemplo, para dividir tu código). Dado que estas tareas no están verificadas por un programa, puedes hacerlo siempre y cuando entregues los archivos obligatorios.
- A veces, las pautas de un ejercicio parecen cortas pero los ejemplos pueden mostrar requisitos que no están escritos explícitamente en las instrucciones.
- ¡Lee cada módulo completo antes de empezar! ¡De verdad, hazlo!

¡Por Odín, por Thor! ¡Usa tu cerebro!



Respecto al Makefile para proyectos C++, se aplican las mismas reglas que en C (ver el capítulo de Normas sobre el Makefile).



Tendrás que implementar muchas clases. Esto puede parecer tedioso, a menos que puedas programar tu editor de texto favorito.



Se le da una cierta cantidad de libertad para completar los ejercicios.

Sin embargo, sigue las reglas obligatorias y no seas perezoso.

¡Te pierdes mucha información útil! No dudes en leer sobre...

conceptos teóricos.

Capítulo III

Instrucciones de IA

- Context

Este proyecto está diseñado para ayudarle a descubrir los componentes fundamentales de su entrenamiento 42.

Para anclar adecuadamente los conocimientos y habilidades clave, es esencial adoptar un enfoque reflexivo al utilizar las herramientas y el soporte de IA.

El verdadero aprendizaje fundamental requiere un esfuerzo intelectual genuino, a través del desafío, la repetición y los intercambios de aprendizaje entre pares.

Para obtener una descripción más completa de nuestra postura sobre la IA (como herramienta de aprendizaje, como parte de la capacitación 42 y como una expectativa en el mercado laboral), consulte las preguntas frecuentes específicas en la intranet.

- Mensaje principal

Construye bases sólidas sin atajos.

Desarrollar realmente habilidades técnicas y de potencia.

Experimente el verdadero aprendizaje entre pares, comience a aprender a aprender y a resolver nuevos problemas.

El recorrido de aprendizaje es más importante que el resultado.

Conozca los riesgos asociados con la IA y desarrolle prácticas de control efectivas y contramedidas para evitar errores comunes.

- Reglas del alumno:

- Debes aplicar el razonamiento a las tareas asignadas, especialmente antes de recurrir a la IA.

- No debes pedir respuestas directas a la IA.
- Debes conocer los 42 enfoques globales sobre IA.

• Resultados de la fase:

Dentro de esta fase fundamental, obtendrás los siguientes resultados:

- Obtenga bases técnicas y de codificación adecuadas.
- Conozca por qué y cómo la IA puede ser peligrosa durante esta fase.

• Comentarios y ejemplo:

Sí, sabemos que la IA existe, y sí, puede resolver tus proyectos. Pero estás aquí para aprender, no para demostrar que la IA ha aprendido. No pierdas tu tiempo (ni el nuestro) solo para demostrar que la IA puede resolver el problema.

Aprender a los 42 años no se trata de saber la respuesta, sino de desarrollar la capacidad de encontrarla. La IA te da la respuesta directamente, pero eso te impide desarrollar tu propio razonamiento. Y razonar requiere tiempo, esfuerzo e implica fracaso. El camino al éxito no se supone que sea fácil.

- Tenga en cuenta que durante los exámenes, la IA no está disponible (no hay Internet, ni teléfonos inteligentes, etc.). Rápidamente se dará cuenta si ha confiado demasiado en la IA en su proceso de aprendizaje.

El aprendizaje entre pares te expone a diferentes ideas y enfoques, mejorando tus habilidades interpersonales y tu capacidad de pensar de forma divergente. Esto es mucho más valioso que simplemente chatear con un bot. Así que no seas tímido: ¡habla, pregunta y aprende con otros!

Sí, la IA formará parte del plan de estudios, tanto como herramienta de aprendizaje como tema en sí mismo. Incluso tendrás la oportunidad de crear tu propio software de IA. Para obtener más información sobre nuestro enfoque progresivo, consulta la documentación disponible en la intranet.

Buenas prácticas:

Estoy atascado con un nuevo concepto. Le pregunto a alguien cercano cómo lo abordó. Hablamos durante 10 minutos y, de repente, todo encaja. Lo entiendo.

Mala práctica:

Uso IA en secreto y copio código que parece correcto. Durante la evaluación por pares, no puedo explicar nada. Suspenso. Durante el examen, sin IA, me quedo atascado de nuevo. Suspenso.

Capítulo IV

Ejercicio 00: Polimorfismo

	Ejercicio: 00
Polimorfismo	
Directorio: ex00/	
Archivos a enviar: Makefile, main.cpp, *.cpp, *.h, hpp}	
Prohibido: Ninguno	

Para cada ejercicio, debes proporcionar las pruebas más completas que puedas. Los constructores y destructores de cada clase deben mostrar mensajes específicos. No utilice el mismo mensaje para todas las clases.

Comience implementando una clase base simple llamada Animal. Tiene un objeto protegido. atributo:

- tipo `std::string`;

Implementar una clase Perro que herede de Animal.

Implementa una clase Gato que herede de Animal.

Estas dos clases derivadas deben establecer su campo de tipo según su nombre. Entonces, el tipo del perro se inicializará como "Perro" y el del gato como "Gato".

El tipo de la clase Animal puede dejarse vacío o establecerse en el valor de su elección.

Cada animal debe poder utilizar la función miembro: `makeSound()`

Imprimirá un sonido apropiado (los gatos no ladran).

Al ejecutar este código debería imprimir los sonidos específicos de las clases Perro y Gato, no los del Animal.

```
int principal()
{
    const Animal* meta = nuevo Animal(); const
    Animal* j = nuevo Perro(); const Animal*
    i = nuevo Gato();

    std::cout << j->getType() << << std::endl; std::cout << i->getType()
    << << std::endl; i->makeSound(); //emitirá el sonido del gato! j-
    >makeSound(); meta->makeSound();

    ...

    devuelve 0;
}
```

Para asegurarte de que entendiste su funcionamiento, implementa una clase WrongCat que herede de una clase WrongAnimal . Si reemplazas el Animal y el Gato por los incorrectos en el código anterior, WrongCat debería emitir el sonido WrongAnimal.

Implementar y entregar más pruebas que las mencionadas anteriormente.

Capítulo V

Ejercicio 01: No quiero incendiar el mundo

	Ejercicio: 01
No quiero incendiar el mundo	
Directorio: ex01/	
Archivos a enviar: Archivos del ejercicio anterior + *.cpp, *.h, *.hpp	
Prohibido: Ninguno	

Los constructores y destructores de cada clase deben mostrar mensajes específicos.

Implementa una clase `Brain` . Contiene un array de 100 `std::string` llamado `ideas`. De esta manera, `Perro` y `Gato` tendrán un atributo `Cerebro*` privado. Durante la construcción, el `Perro` y el `Gato` crearán su `Cerebro` usando `new Brain()`; durante la destrucción, el `Perro` y el `Gato` eliminarán su `Cerebro`.

En la función principal, crea y completa un array de objetos `Animal` . La mitad serán objetos `Perro` y la otra mitad, objetos `Gato` . Al finalizar la ejecución del programa, recorre este array y elimina todos los objetos `Animal`. Debes eliminar directamente perros y gatos como `Animales`. Los destructores correspondientes deben llamarse en el orden previsto.

No olvides comprobar si hay fugas de memoria.

Una copia de un perro o un gato no debe ser superficial. Por lo tanto, ¡debes comprobar que tus copias sean profundas!

```
int principal()
{
    const Animal* j = nuevo Perro(); const
    Animal* i = nuevo Gato();

    eliminar j;//no debería crear una fuga eliminar i;

    ...

    devuelve 0;
}
```

Implementar y entregar más pruebas que las mencionadas anteriormente.

Capítulo VI

Ejercicio 02: Clase abstracta

	Ejercicio: 02
Clase abstracta	
Directorio: ex02/	
Archivos a enviar: Archivos del ejercicio anterior + *.cpp, *.h, *.hpp	
Prohibido: Ninguno	

Crear objetos animales no tiene sentido después de todo. ¡Es cierto, no hacen ruido!

Para evitar posibles errores, la clase Animal predeterminada no debe ser instanciable. Corrige la clase Animal para que nadie pueda instanciarla. Todo debería funcionar como antes.

Si lo desea, puede actualizar el nombre de la clase agregando un prefijo A a Animal.

Capítulo VII

Ejercicio 03: Interfaz y resumen

	Ejercicio: 03
Interfaz y resumen	
Directorio: ex03/	
Archivos a enviar: Makefile, main.cpp, *.cpp, *.h, *.hpp	
Prohibido: Ninguno	

Las interfaces no existen en C++98 (ni siquiera en C++20). Sin embargo, las clases puramente abstractas se denominan comúnmente interfaces. Por lo tanto, en este último ejercicio, intentaremos implementar interfaces para asegurarnos de que comprendes este módulo.

Complete la definición de la siguiente clase AMateria e implemente las funciones miembro necesarias.

```

class AMateria
{
    protegido: [...]

    público:
        AMateria(std::string const & tipo); [...]

        std::string const & getType() const; //Devuelve el tipo de materia

        virtual AMateria* clone() const = 0; virtual void
        use(ICharacter& target);
};

```

Implementa las clases concretas para las Materias: Hielo y Cura. Usa sus nombres en minúsculas ("hielo" para Hielo, "cura" para Cura) para definir sus tipos. Por supuesto, su función miembro clone() devolverá una nueva instancia del mismo tipo (es decir, si clonas una Materia de Hielo, obtendrás una nueva Materia de Hielo).

La función miembro use(ICCharacter&) mostrará:

- Hielo: "*" dispara un rayo de hielo a <nombre> **"
- Cura: "*" cura las heridas de <nombre> **"

<nombre> es el nombre del carácter pasado como parámetro. No se imprimen los corchetes angulares (< y >).



Al asignar una Materia a otra, copiar el tipo no hace sentido.

Escriba la clase concreta Character que implementará la siguiente interfaz:

```
class ICarácter {

    público:
        virtual ~ICarácter() {} virtual std::string const &
            getName() const = 0; virtual void equipar(AMateria* m) = 0; virtual void desequipar(int
            idx) = 0; virtual void usar(int idx, ICarácter& objetivo) = 0;

};
```

El Personaje posee un inventario de 4 ranuras, lo que significa como máximo 4 Materias. El inventario está vacío al construir. Equipan las Materias en el primer espacio vacío que encuentren, en el siguiente orden: del espacio 0 al 3. Si intentan añadir una Materia a un inventario lleno, o usar/desequipar una Materia inexistente, no debería ocurrir nada (pero los errores siguen estando prohibidos). ¡La función miembro unequip() NO debe eliminar la Materia!



Manipula las Materias que tu personaje deja en el suelo como quieras. Guarde las direcciones antes de llamar a unequip(), o cualquier otra cosa, pero no olvide que debe evitar pérdidas de memoria.

La función miembro use(int, ICharacter&) tendrá que usar Materia en el slot[idx] y pasa el parámetro de destino a la función AMateria::use.



El inventario de tu personaje podrá soportar cualquier tipo de
Una materia.

Tu personaje debe tener un constructor que tome su nombre como parámetro. Cualquier copia (mediante un constructor de copias o un operador de asignación de copias) de un personaje debe ser profunda. Durante la copia, las Materias de un Personaje deben eliminarse antes de añadir las nuevas a su inventario. Por supuesto, las Materias deben eliminarse al destruir un Personaje.

Escriba la clase concreta `MateriaSource` que implementará la siguiente interfaz:

```
clase IMateriaSource
{
    público:
        virtual ~IMateriaSource() {} virtual void
            learnMateria(AMateria*) = 0; virtual AMateria*
            createMateria(std::string const & type) = 0;
};
```

- `aprenderMaterial(AMateria*)`

Copia la Materia pasada como parámetro y la almacena en memoria para que pueda clonarse posteriormente. Al igual que el Personaje, la `MateriaSource` puede conocer un máximo de 4 Materias. No son necesariamente únicas.

- `crearMateria(std::string const &)`

Devuelve una nueva Materia. Esta es una copia de la Materia previamente aprendida por `MateriaSource`, cuyo tipo es igual al pasado como parámetro. Devuelve 0 si se desconoce el tipo.

En resumen, su `MateriaSource` debe ser capaz de aprender "plantillas" de Materias para crearlas cuando sea necesario. Luego, podrá generar una nueva Materia usando simplemente una cadena que identifique su tipo.

Ejecutando este código:

```
int principal() {  
  
    IMaterialSource* src = nuevo MaterialSource(); src->  
    >learnMatter(nuevo Hielo()); src->  
    >learnMaterial(nuevo Cura());  
  
    ICaracter* yo = nuevo Personaje("yo");  
  
    AMateria* tmp; tmp  
    = src->createMateria("hielo"); me->equip(tmp);  
    tmp = src->  
    >createMateria("cura"); me->equip(tmp);  
  
    PersonajeIcaracter* bob = nuevo Personaje("bob");  
  
    yo->uso(0, *bob); yo->  
    >uso(1, *bob);  
  
    eliminar bob;  
    eliminarme ;  
    eliminar src;  
  
    devuelve 0;  
}
```

Debería mostrar:

```
$> clang++ -W -Wall -Werror *.cpp $> ./a.out | cat  
-e * dispara un rayo de hielo  
a Bob *$ * cura las heridas de Bob *$
```

Como de costumbre, implemente y entregue más pruebas que las indicadas anteriormente.



Puedes aprobar este módulo sin realizar el ejercicio 03.

Capítulo VIII

Presentación y evaluación por pares

Entrega tu tarea en tu repositorio de Git como de costumbre. Solo se evaluará el trabajo dentro de tu repositorio durante la defensa. No dudes en revisar los nombres de tus carpetas y archivos para asegurarte de que sean correctos.

Durante la evaluación, ocasionalmente se podría solicitar una breve modificación del proyecto. Esto podría implicar un cambio menor de comportamiento, la escritura o reescritura de algunas líneas de código, o una función fácil de añadir.

Si bien este paso puede no ser aplicable a todos los proyectos, debes estar preparado para ello si se menciona en las pautas de evaluación.

Este paso tiene como objetivo verificar su comprensión real de una parte específica del proyecto. La modificación se puede realizar en cualquier entorno de desarrollo que elija (por ejemplo, su configuración habitual) y debería ser factible en unos pocos minutos, a menos que se defina un período de tiempo específico como parte de la evaluación.

Por ejemplo, se le puede pedir que realice una pequeña actualización a una función o script, modifique una pantalla o ajuste una estructura de datos para almacenar nueva información, etc.

Los detalles (alcance, objetivo, etc.) se especificarán en las pautas de evaluación y pueden variar de una evaluación a otra para el mismo proyecto.